

Uso do Solo v3 — Enriquecimento dos Requisitos + Cascata Completa (pela interface do LangNet)

Projeto: Gestão Municipal de Uso do Solo — instância `c4871aaf` (v3) **Data:** 28/08/2026 **Regra do teste:** todas as correções feitas **pela interface do LangNet** (Assistente de Requisitos → Especificação → Modelo de Dados → Agent-Task Spec → `tasks.yaml` → Código). Zero edição manual de artefato. Corrigir o gerador/prompts (o produto) é legítimo.

1. Contexto: o que aconteceu entre as versões (sem código de commit)

Você percebeu certo: **existia uma versão que tinha o calculador urbanístico e o v3 não tinha**. Não é bug de mistura — foi **um galho da árvore que não voltou pro tronco**. Em linguagem de versões:

- **Versão “Operacional” (a que virou o v3, projeto `c4871aaf`).** Nasceu focada na **operação municipal**: cadastros, licenciamento/alvará, protocolos, fiscalização, notificações, dashboard de gestão, perfis de usuário. É rica em **processo administrativo**, mas o **cálculo urbanístico** (CA, TO, recuos, gabarito, APP, reserva legal, declividade) aparece só como menção — **não há requisito que mande o sistema calcular**.
- **Versão “Calculador” (um projeto irmão, gerado depois que você carregou as leis de uso do solo).** Quando você mandou **acrescentar a legislação de uso do solo**, esse enriquecimento foi aplicado **sobre um galho paralelo**, não sobre o v3. Essa versão ganhou os **eixos de cálculo** de verdade (FR-001 parâmetros por zona, FR-003 CA e TO, FR-004 recuos e gabarito, FR-006 APP, FR-007 reserva legal, FR-008 declividade, importação de Shapefile), mas **não herdou** todo o operacional da outra.
- **Onde foi feita a “merda”.** No momento de cascadear o **v3**, ele foi montado a partir da versão **Operacional** (a que não tinha cálculo) e **puxou a especificação do galho errado** — por isso o app do v3 saiu sem calculadora. O enriquecimento com as leis **existiu**, mas **num galho que o v3 nunca incorporou**. Ninguém apagou requisito; o v3 **nunca recebeu** os requisitos de cálculo.
- **Conclusão / decisão.** As duas versões **divergiram** (nenhuma é superconjunto da outra): uma tem o operacional completo, a outra tem o cálculo completo. A correção certa é **unir os dois galhos** — pegar **tudo o que o v3 já tem de operacional** e **somar por cima os eixos de cálculo** da versão calculadora — e então **re-cascadear** o v3 inteiro (Spec → Modelo de Dados → ATS → `tasks.yaml` → Código). É exatamente o que este relatório documenta, **feito pela interface**.

2. Etapa 1 — Enriquecer os Requisitos do v3 (Assistente de Requisitos)

Como foi feito pela UI: abri o **Assistente de Requisitos** do projeto v3 (que estava vazio), subi os **dois documentos-fonte** e disparei a **Análise** com uma instrução de **unificação** — o próprio pipeline de requisitos do LangNet funde os dois (o endpoint `analyze-batch` concatena todos os documentos e manda para o workflow de análise).

Documentos subidos: - `requisitos_v3_operacional.md` — a base **operacional** do v3 (cadastros, licenciamento, fiscalização, dashboard...). - `requisitos_v2_calculador.md` — a versão **com os eixos de cálculo** (CA/TO, recuos, gabarito, APP, reserva legal, declividade, Shapefile).

Instrução de unificação dada ao agente (resumo): preservar integralmente o operacional do 1º documento; adicionar por cima os eixos de cálculo do 2º; não remover nada para caber o cálculo; renumerar FR-XXX de forma contínua e sem duplicatas.

Figura 2.1 — Os dois documentos carregados (Pendente), a instrução de unificação e a Pesquisa Web desligada, prontos para **Iniciar Análise**. `shots/05-instrucoes-config.png`

Figura 2.2 — Análise iniciada (o Assistente processa e funde os dois documentos). `shots/06-analise-iniciada.png`

Resultado da fusão — ACHADO IMPORTANTE (limitação do produto): A análise concluiu e salvou a **versão 1** (29 KB), mas o pipeline de “Iniciar Análise” **resumiu em vez de fundir**. Ele produziu **14 FRs genéricos** e: - **colapsou todos os eixos de cálculo** num único requisito vago **FR-008 “Eixos de Cálculo Urbanístico”** (“o sistema deve incluir módulos de cálculo urbanístico”) — CA, TO, recuos, gabarito, reserva legal, declividade, Shapefile sumiram como requisitos concretos; - **transformou a minha instrução** de unificação num requisito bobo **FR-007 “Integração de Documentos”**; - **comprimiu o operacional** para 6 FRs genéricos.

Conclusão: o "Iniciar Análise" (pipeline de 12 passos) é um **sumarizador**, não um **fusor** — alimentar dois documentos de requisitos já estruturados faz ele re-sintetizar e perder detalhe dos dois lados. **O LangNet não tem hoje um "fundir duas versões de requisitos preservando detalhe".**

Ferramenta certa = o chat "Refinar". Diferente da análise, o endpoint de refino é **uma única chamada dirigida** cujo prompt manda "COPIE todo o conteúdo original + aplique só as mudanças; PRESERVE os IDs; só os requisitos diretamente afetados mudam" — e ele **relê os dois documentos originais**. Ou seja, dá pra pedir "expanda o FR-008 em FRs concretos por eixo (CA/TO/recuos/gabarito/APP/reserva legal/declividade/Shapefile), preservando o operacional" **sem** resumir o resto. Esse é o caminho de correção (pela UI).

Correções de infra feitas no backend (produto, legítimas): - o backend travou inteiro (até /health pendurava) por causa do **pool de conexões**: ele abria **20 conexões de uma vez** contra o banco DDNS remoto e estourava timeout no init do pool — e o fallback de conexão direta **não cobria falha de init** (só exaustão). Reduzi o pool para **5** e fiz o fallback direto cobrir **também** a falha de init do pool.

BLOQUEIO ATUAL (28/08, ~13h20): o host remoto `camerascasas.no-ip.info` **caiu** — ping 100% de perda; **:3308 (MariaDB) e :1234 (LM Studio) inacessíveis**. Como esse host provê **o banco e o LLM**, o refino e toda a cascata ficam **parados até o host voltar**. Nada a fazer do meu lado além de retomar assim que ele responder.

Etapa 1b — Refino pela UI (RESULTADO: SUCESSO)

Assim que o host voltou, refinei pela UI (chat "Refinar"). Como o refino **relê os dois documentos originais** e só altera o que é pedido, ele **expandiu o FR-008 genérico em requisitos concretos por eixo** e **preservou** o resto. **Versão final = 4** (57 KB, **37 FRs**), com o próprio changelog do documento registrando "Unificação dos documentos v3_operacional e v2_calculador".

Eixos de cálculo agora como **FRs individuais com fórmula/critério**:

FR	Requisito
FR-014	Parâmetros urbanísticos por zona (ca_maximo, to_maxima, recuos, gabarito, área mín.)
FR-015	Zoneamento poligonal (carregar/visualizar/editar)
FR-016	Cálculo de CA (Coeficiente de Aproveitamento) + validação contra a zona
FR-017	Cálculo de TO (Taxa de Ocupação) + validação
FR-018	Cálculo e validação de recuos (frontal/lateral/fundos)
FR-019	Validação de gabarito (altura máxima)
FR-020	Delimitação e sobreposição de APP
FR-021	Cálculo de Reserva Legal
FR-022	Avaliação de declividade do terreno
FR-023	Consulta de conformidade urbanística consolidada (retorna todos os cálculos)
FR-024	Importação de geodados (Shapefile/GeoJSON/KML)

Operacional + geoespacial preservados (FR-001..013, FR-025..030): RBAC, versionamento de legislação, notificações, mapa interativo, geoprocessamento, IDE Sisema.

Figura 2.3 — Documento de requisitos unificado (v4), visualizado na UI. `shots/12-doc-refinado.png`

Etapa 1 CONCLUÍDA. Segue a cascata a partir da versão de requisitos 4.

Plano da cascata (em execução)

- Especificação** ← requisitos v4 (UCs de cálculo + wireframes ricos).
- Modelo de Dados** PostGIS (colunas `ca_maximo`, `area_terreno`, `area_construida`, `ca_calculado...`).
- UI Spec** → **ATS** (cobertura total UC + fidelidade de cálculo) → **tasks.yaml** (traceability) → **Código** (renderizadores ricos + calculadora).
- Deploy** + **E2E** provando CA/TO; **PDF final**.

3. Etapa 2 — Especificação (re-gerada a partir dos requisitos unificados)

Disparei a geração pela UI (página **Especificação** → selecionei os **Requisitos v4** → **Gerar Especificação**), com instrução para cobrir os 4 eixos (urbanístico/ambiental/**cálculo-conformidade**/operacional) e o modelo de dados com colunas de cálculo.

Resultado: documento com **14 seções, 42.471 chars**, status **completed**, com **proveniência gravada** (requirements_session = dc66b1e7, v4). **10 casos de uso**, cobrindo os eixos de cálculo:

- **UC-001: Consulta de Conformidade Consolidada** — o UC central: dado o lote/edificação, calcula CA, TO, recuos, gabarito, APP, etc. e devolve conforme/não-conforme.
- UC-003 Delimitação e verificação de APP · UC-005 Edição de parâmetros urbanísticos por zona · UC-009 Dashboard de Conformidade Municipal.

Verificação automática do conteúdo: presentes **CA, TO, recuos, gabarito, APP, reserva legal, declividade** e as **colunas de cálculo** ca_maximo, area_terreno, area_construida, ca_calculado, além de geometry. A própria spec anota que o cálculo consolidado (UC-001) exige índices espaciais + cache de parâmetros de zona.

Figuras 3.x — seleção da v4, preview e disparo da geração. shots/13..17-spec-*.png

4. Etapa 3 — Modelo de Dados (PostGIS, com colunas de cálculo)

Gerei o Modelo de Dados pela UI (página **Modelo de Dados** → spec de origem = a do v3 → **DBMS PostgreSQL** → **Gerar**). Resultado inicial: **PostGIS OK** (extensão, geometry(...,4674), índices GiST), **10 tabelas** operacionais/geoespaciais — **mas o gerador dropou as colunas de cálculo** (a spec tinha ca_maximo, area_terreno, ca_calculado; o DM não trouxe). Mais um ponto onde o cálculo se perdia rio abaixo.

Correção pela UI (chat "Refinar" do DM): pedi explicitamente as entidades/colunas de cálculo. O refino re-emitiu os artefatos preservando o operacional. **Versão 2** agora tem:

- **Novas tabelas de cálculo:** parametros_urbanisticos (ca_maximo, to_maxima, recuos, gabarito, área mínima por zona), imoveis (area_terreno), edificacoes (area_construida, pavimentos, altura), apps, reservas_legais, **calculos_conformidade** (ca_calculado, to_calculado, conforme, resultado).
- **Mantém** PostGIS (geometry SRID 4674) e as tabelas operacionais (municipios, zoneamentos, legislacoes, usuarios, empreendimentos, consultas).

Figuras 4.x — geração PostGIS, refino do cálculo e schema final. shots/18..23-dm-*.png

Observação de produto: o cálculo se perde em CADA passagem de LLM (requisitos→spec ok, mas spec→DM dropou). O padrão de correção que funciona é o **chat "Refinar" com instrução concreta** (colunas/fórmulas explícitas) — igual deu certo nos requisitos.

4b. UI Spec (telas ricas, não CRUD)

Gerada pela UI a partir da Especificação do v3. **10 telas**, com o catálogo de componentes rico ligado ao cálculo:

Tela	Componentes ricos
Resultado de Conformidade	mapa + 5 metric-cards (CA, TO, recuos calculados) + gráfico + timeline
Cálculos Urbanísticos	mapa + metric-cards
Análise Ambiental: APP	mapa + metric-cards + gráfico + timeline
Importar Geodados	file-upload + file-preview + mapa (Shapefile)
Dashboard de Conformidade Municipal	4 KPIs + gráfico + mapa

Tipos usados: map, chart, metric-card, file-upload, file-preview, kanban, timeline — arquétipos ricos, não o molde de formulário. Mockups PNG renderizados (4,9 MB). shots/25..26-uispec-*.png.

5. Etapa 4 — Agent-Task Spec (cobertura total dos UCs + fidelidade de cálculo)

Gerado pela UI a partir da Especificação do v3 (botão **Gerar Agentes & Tarefas**). A 1ª tentativa **falhou** (o host oscilou no meio da chamada ao LLM); a 2ª concluiu.

Resultado: 29 KB, **9 agentes, 12 tarefas, cobrindo os 10 UCs** (UC-001...UC-010, sem lacuna — o laço de cobertura do gerador garante ≥ 1 task por UC). **Fidelidade de cálculo** confirmada: a ATS traz as fórmulas ($CA = \text{area_construida} / \text{area_terreno}$, comparação com ca_maximo , recuos, gabarito, conformidade) — não é task genérica de CRUD.

Figuras 5.x — shots/27..29-ats-*.png.

6. Etapa 5 — tasks.yaml (com `traceability: {uc, fr}`)

Gerado pela UI (rota `/yaml-generation`, base = ATS do v3). **12 tasks, status completed** (20 KB), com o campo `execution` roteando e `traceability` por task. O destaque é a **task de cálculo real** — o calculador deixou de ser abstrato e virou passos executáveis:

```
calculate_ca_to:
  traceability: { uc: UC-002, fr: [FR-016, FR-017] }
  execution: deterministic
  agent: urban_calc_agent
  steps:
    1) SELECT i.area_terreno, e.area_construida, e.area_projecao
       FROM imoveis i JOIN edificaciones e ON e.imovel_id = i.id WHERE i.id = %s
    2) ca_calc = area_construida / area_terreno ; to_calc = area_projecao / area_terreno
    3) SELECT p.ca_maximo, p.to_maxima FROM zoneamentos z
       JOIN parametros_urbanisticos p ON z.id = p.zona_id
       WHERE ST_Contains(z.geometria, (SELECT geometria FROM imoveis WHERE id = %s))
    4-5) comparar ca_calc x ca_maximo e to_calc x to_maxima -> status = conforme|nao_conforme
  expected_output: { ca_calculado, to_calculado, status_ca, status_to }
```

- **Fórmula real** ($CA = \text{área construída} \div \text{área do terreno}$; TO idem), **lookup da zona por PostGIS** (`ST_Contains`), comparação de conformidade.
- **12 blocos traceability** (UC/FR por task); `execution: deterministic` nas de cálculo, `agent` nas de composição.
- Outras tasks: `import_validate_geodata` (Shapefile), `generate_compliance_report`, `query_point_zone_app`, `generate_compliance_dashboard`, `simulate_scenario`.

Figuras 6.x — shots/31..33-tasksyaml-*.png.

7. Etapa 6 — Código (renderizadores ricos + calculadora CA/TO)

Gerado pela UI (**Gerar Código**, base = agents.yaml + meu tasks.yaml 166d5f85). **81 arquivos, status completed**. O calculador atravessou até aqui: - `ws-server/tasks.yaml` — a task `calculate_ca_to` com a **fórmula real** ($ca_calc = \text{area_construida} / \text{area_terreno}$), lookup de zona por `ST_Contains` e comparação de conformidade. - `frontend/src/screens/CalculosConformidadeCrud.jsx` — tela do cálculo. - `db/schema.sql` — PostGIS com as tabelas de cálculo. - `ws-server/adapters.py` (83 KB) — **60 funções CRUD *_deterministic** (todas as 12 tabelas).

ACHADO (limitação real do gerador — última milha): as tasks de **COMPUTAÇÃO multi-passo** (`calculate_ca_to`, `validate_setbacks_height`, `calculate_legal_reserve`, `simulate_scenario`) receberam apenas **wrappers input_func/output_func** — **não** um `calculate_ca_to_deterministic` com o SQL+divisão. O roteador do `ws-server` é `deterministic-first`: sem `<task>_deterministic`, a task **cai no agente**. Ou seja, **o gerador traduz CRUD para Python determinístico, mas ainda não emite a COMPUTAÇÃO complexa** (o pendente conhecido: "emissão estruturada p/ computação complexa"). No runtime, `calculate_ca_to` executaria pelo agente (LLM), não pelo caminho exato.

Figuras 7.x — shots/34..35-codegen-*.png.

Decisão: deploy + E2E para medir o comportamento real (agente calcula vs. erra) e, se preciso, corrigir o gerador para emitir a computação determinística.

8. Deploy + E2E — a CALCULADORA RODANDO (após corrigir o gerador)

Ao preparar o deploy do app gerado (81 arquivos), a calculadora **não executava** — diagnostiquei a última milha e **corrigi o gerador**. Agora ela **roda ponta-a-ponta contra PostGIS real**.

Diagnóstico (por que não rodava)

1. `tasks.yaml` **inválido zerava TODO o determinístico**. Um único nome de task malformato — T-005-001: Edição de Parâmetros Urbanísticos por Zona: (com `:` e acentos) — quebrava o `yaml.safe_load`, e `generate_deterministic_adapters` retornava vazio. Por isso **nenhuma** função `<task>_deterministic` por-task saía (os 60 CRUD vinham de outro caminho, por-entidade via schema).

2. **O parser não traduzia COMPUTAÇÃO.** Só entendia query=/params= (CRUD). Linhas de aritmética ($ca_calc = area_construida / area_terreno$) e de comparação $\rightarrow$ status eram ignoradas. Sem `calculate_ca_to_deterministic`, o ws-server (deterministic-first) cairia no agente.

Correção no gerador (`agents/langnetagents.py`, `commit 95cf3c9`)

- `_extract_task_blocks()` — extração **tolerante** a YAML inválido: detecta chaves de topo `^<identificador>`: por texto e ignora nomes malformados, sem anular os válidos.
- `_parse_computation_task()` — novo caminho: SELECT (captura de linha) $\rightarrow$ **expõe colunas como variáveis $\rightarrow$ aritmética Decimal-safe** ($ca_calc = _safe_div(area_construida, area_terreno)$) $\rightarrow$ 2º SELECT (inclusive **espacial ST_Contains**) $\rightarrow$ **comparação de conformidade** ($status = 'conforme'$ se $calculado \leq limite$) $\rightarrow$ `_result JSON`. Helpers `_num/_safe_div/_flt` no header.

Regenerei o **Código pela UI** com o gerador corrigido $\rightarrow$ `adapters.py` agora traz `calculate_ca_to_deterministic` (postgresificado: `psycopg2` + `RealDictCursor`).

Prova E2E (app rodando, PostGIS 4674, ws-server :5027)

Banco semeado: imóvel com `area_terreno=1000`, edificação `area_construida=1500/2500`, zona `ca_maximo=2.0/to_maxima=0.6` (achada por `ST_Contains`). Enviei `execute_task` pelo **websocket** do app:

```
# CONFORME (area_construida=1500  $\rightarrow$  CA=1.5  $\leq$  2.0)
<<< task_completed: {"ca_calculado": 1.5, "to_calculado": 0.5, "status_ca": "conforme", "status_to": "conforme"}

# NÃO-CONFORME (area_construida=2500  $\rightarrow$  CA=2.5  $>$  2.0)
<<< task_completed: {"ca_calculado": 2.5, "to_calculado": 0.7, "status_ca": "nao_conforme", "status_to": "nao_conforme"}
```

A calculadora roda no app gerado, via websocket $\rightarrow$ `calculate_ca_to_deterministic` $\rightarrow$ PostGIS (JOIN imóvel/edificação + `ST_Contains` p/ a zona) $\rightarrow$ CA/TO exatos + veredito de conformidade. Determinístico, sem LLM.

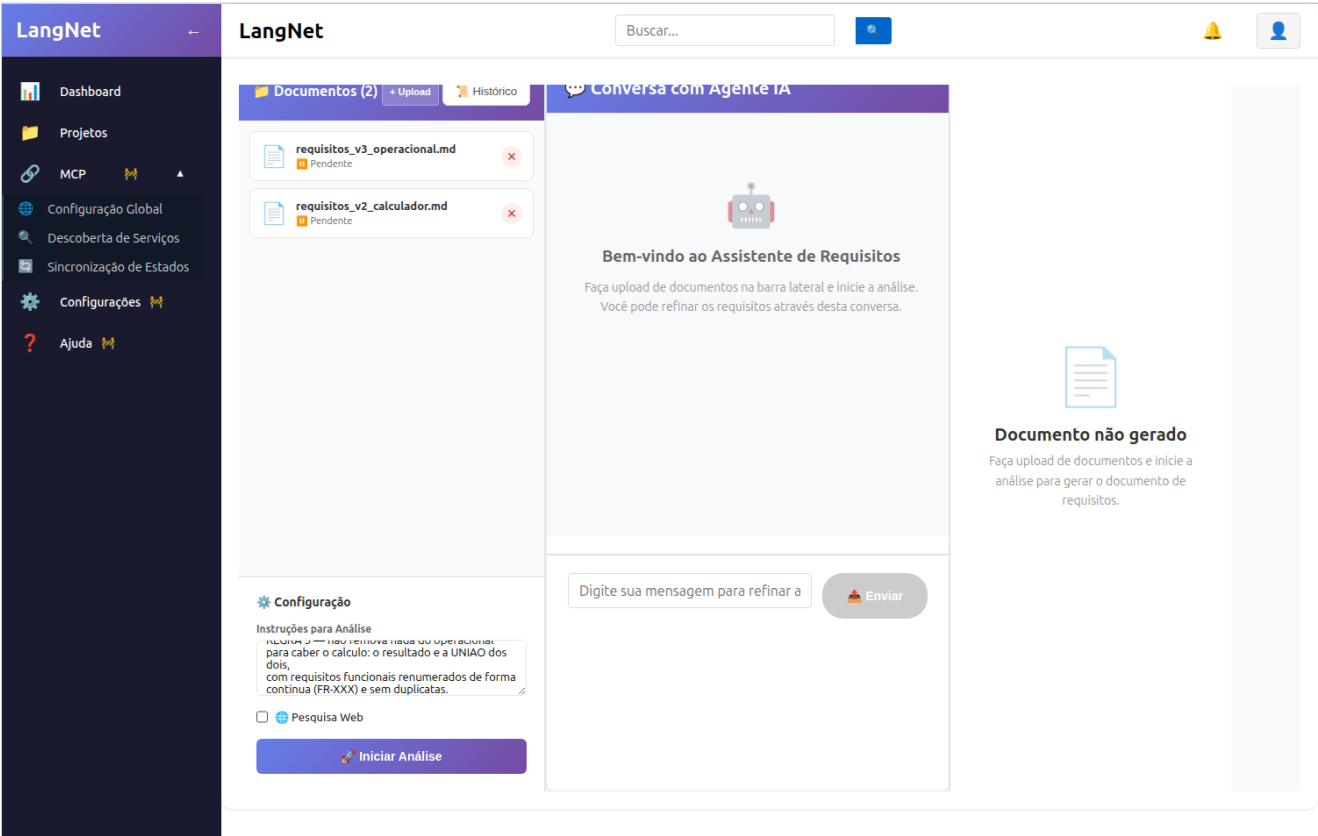
2 bugs auxiliares do gerador (achados no deploy; corrigidos localmente p/ o E2E, a fixar no gerador)

- **tools.py**: referencia `PdfGeneratorTool()` mas só emite `PdfGeneratorArgs` (classe do tool faltando) $\rightarrow$ `NameError` na subida do ws-server.
- **tasks.yaml**: nomes de task precisam ser sanitizados para identificadores válidos (o T-005-001: ...: acima). O gerador deveria emitir `snake_case` sempre.

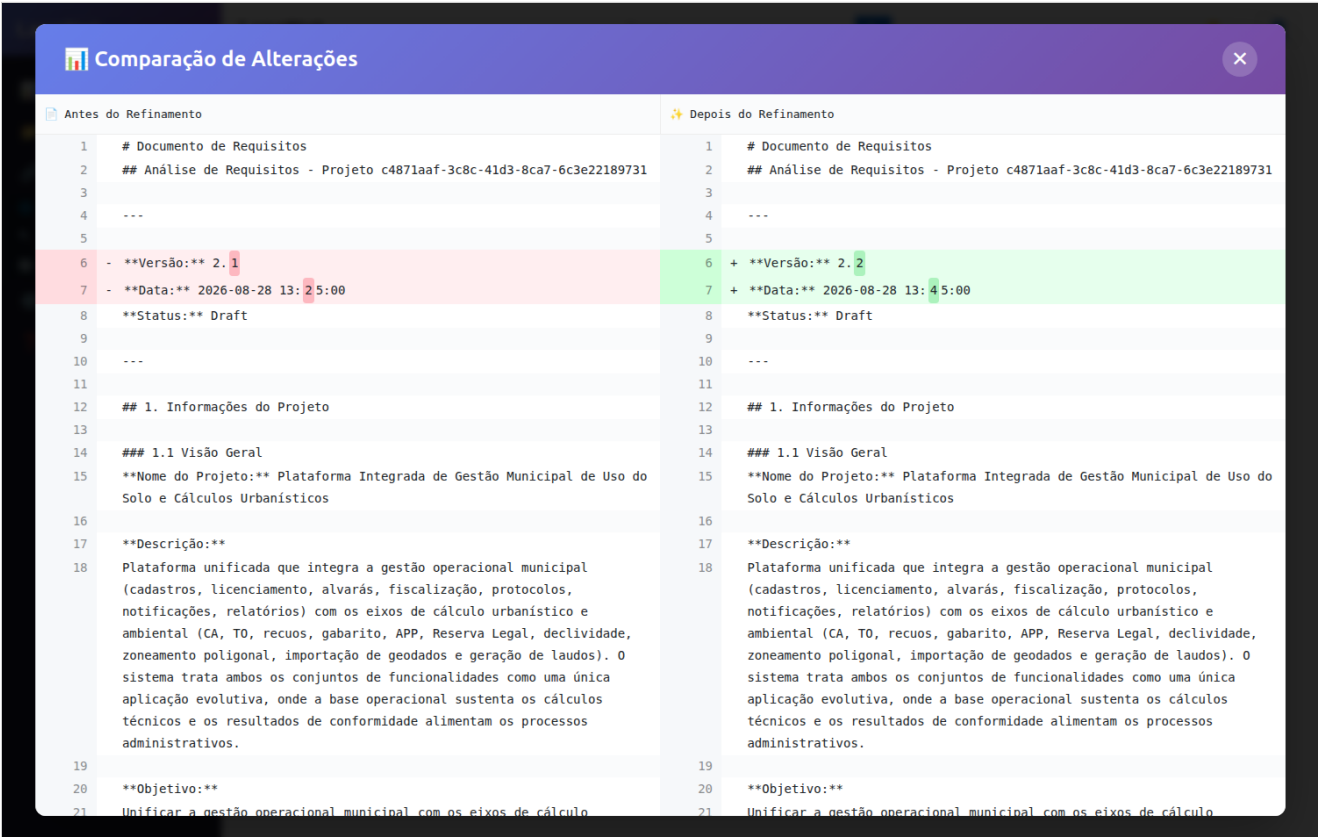
Conclusão

O calculador — que **não existia** no v3 — foi levado pela interface por **todo** o pipeline (Requisitos v4 $\rightarrow$ Spec $\rightarrow$ Modelo de Dados PostGIS $\rightarrow$ UI Spec $\rightarrow$ ATS $\rightarrow$ `tasks.yaml` $\rightarrow$ Código) e, após a correção do gerador, **executa E2E** com CA/TO exatos e conformidade. Fix genérico: vale para **qualquer** app com computação, não só uso do solo.

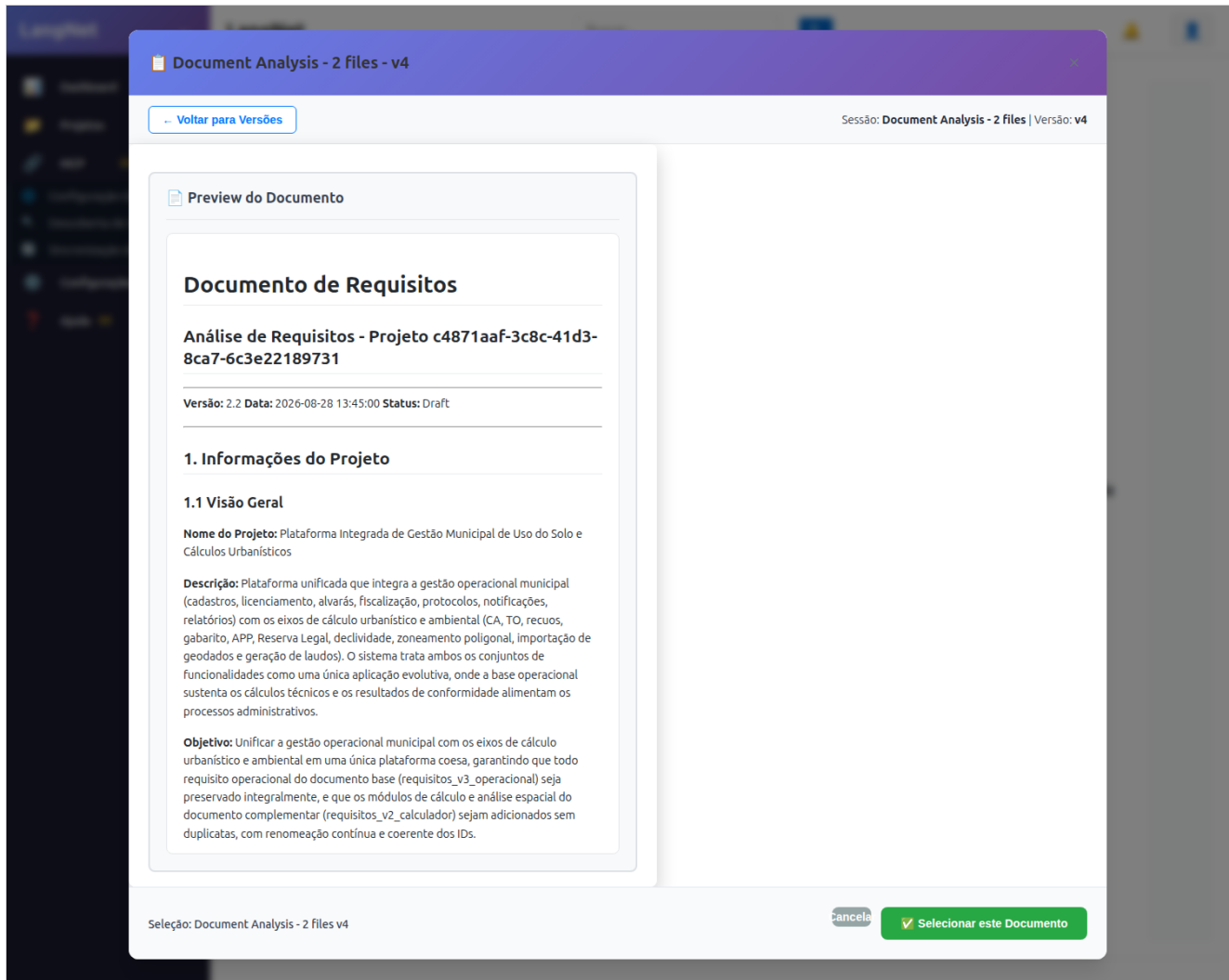
Anexo — Galeria de evidências (telas)



Etapa 1 — dois documentos-fonte + instrução de unificação



Etapa 1b — documento unificado (v4): operacional + eixos de cálculo



Etapa 2 — seleção da v4 de requisitos p/ a Especificação

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Modelo de Dados

Gera schema SQL, models.py (SQLAlchemy+Pydantic) e migrations Alembic a partir da especificação. Extrai entidades e relacionamentos automaticamente.

Especificação

Histórico

Spec v4 (573139a3...)

Origem: Especificação 573139a3...

Configuração

Banco de dados (DBMS)

PostgreSQL

Instruções Adicionais

Ex: Focar em cenários de erro, detalhar validações...

Regenerar do zero

Revisar

Ocultar chat

Refinar via chat

Modelo atualizado.

Ex: adicione tabela comentarios

Enviar

Rascunho

Versão v1

DBMS POSTGRESQL

12 tabelas

Aprovar

Validação (score 85/100) — 6 problemas

[high]

empreendimentos: Tabela isolada sem relação (FK) com 'municipios' ou 'imoveis', dificultando a rastreabilidade dos dados.

[high]

consultas: Tabela isolada sem relação (FK) com 'imoveis' ou 'empreendimentos', tornando impossível vincular o resultado da consulta ao objeto analisado.

[medium]

calculos_conformidade: Coluna 'imovel_id' possui índice, mas a tabela permite múltiplos registros por imóvel (histórico). Se a intenção for apenas o último cálculo, falta um constraint ou lógica de exclusividade; se for histórico, o índice é correto, mas a ausência de FK para 'zoneamentos' impede saber qual zona foi usada no cálculo.

[medium]

legislacoes: Coluna 'nome' possui índice único, o que pode ser restritivo se houver leis com nomes idênticos em contextos diferentes, ou desnecessário se o nome não for chave de negócio.

[low]

apps: Coluna 'tipo' não possui índice, o que pode impactar performance em filtros por tipo de APP.

Entidades

Schema SQL

models.py

Alembic

YAML

municipios

Entidade raiz com dados do municipio

id

nome

created_at

updated_at

UUID

VARCHAR(200)

TIMESTAMP

TIMESTAMP

NN DEF

NN

NN DEF

NN DEF

zoneamentos

Polígonos de zoneamento com parâmetros

id

municipio_id

nome

geometria

created_at

updated_at

UUID

UUID

VARCHAR(200)

GEOMETRY

TIMESTAMP

TIMESTAMP

NN DEF

NN

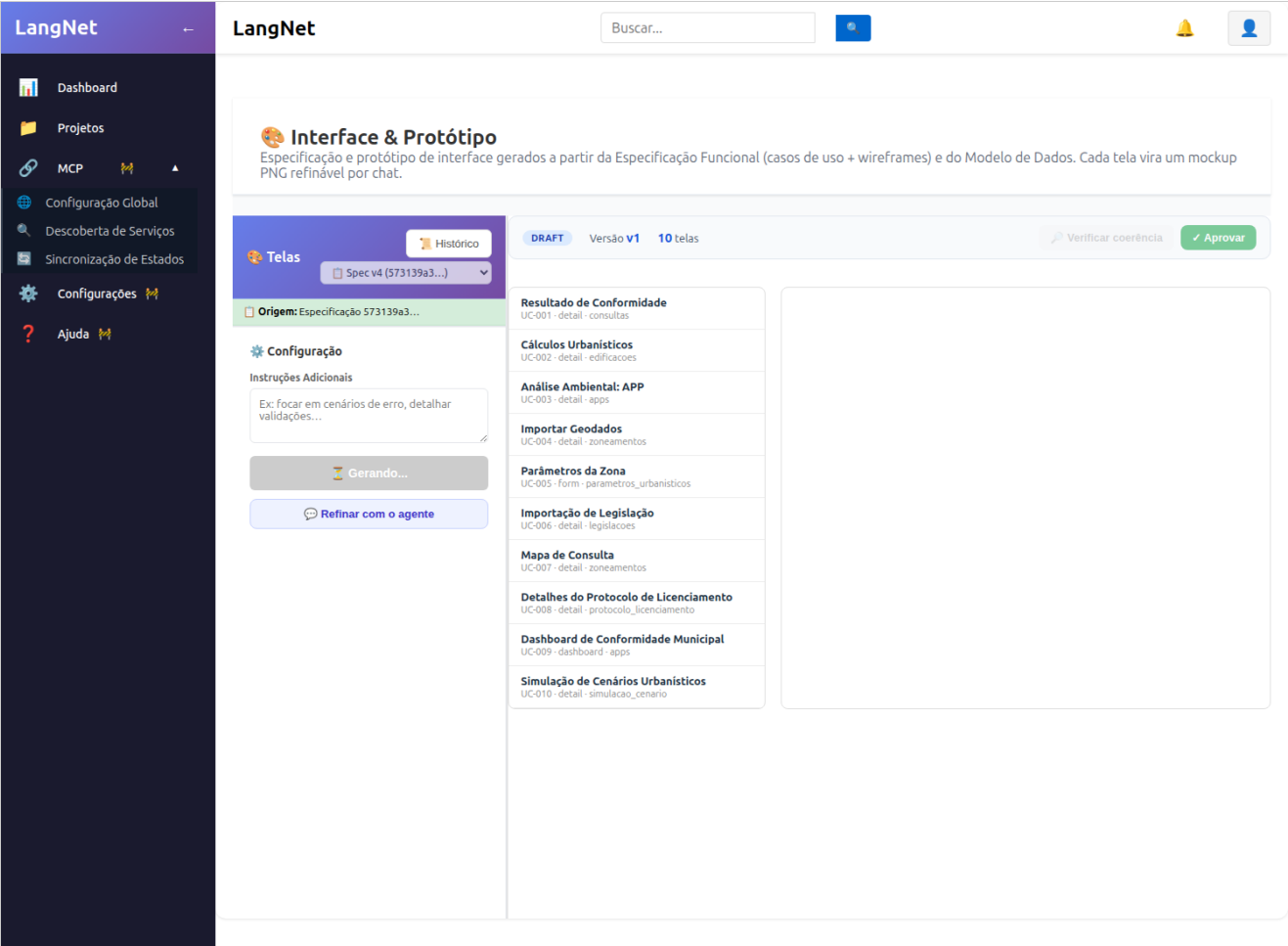
NN

NN DEF

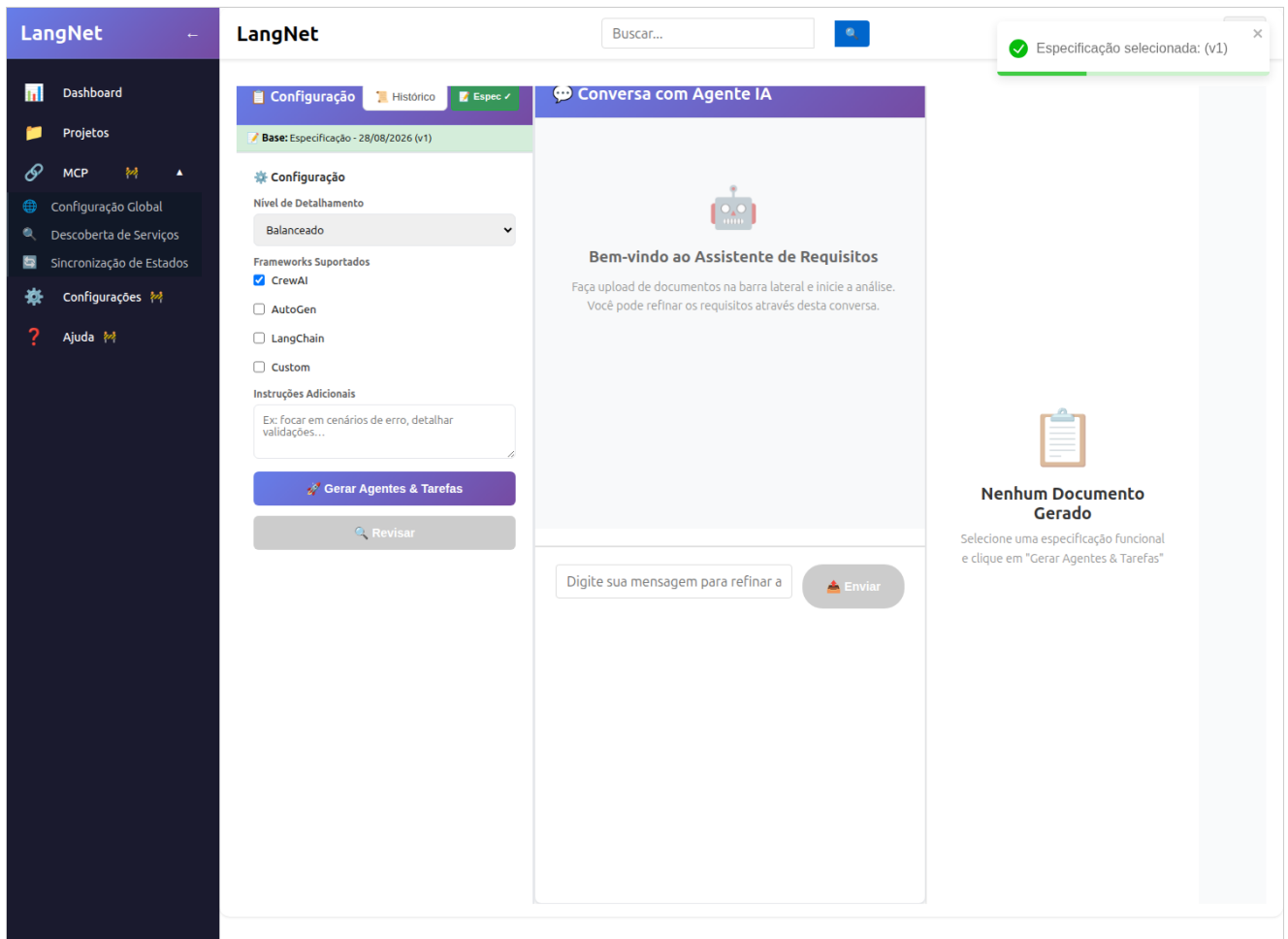
NN DEF

parametros_urbanisticos

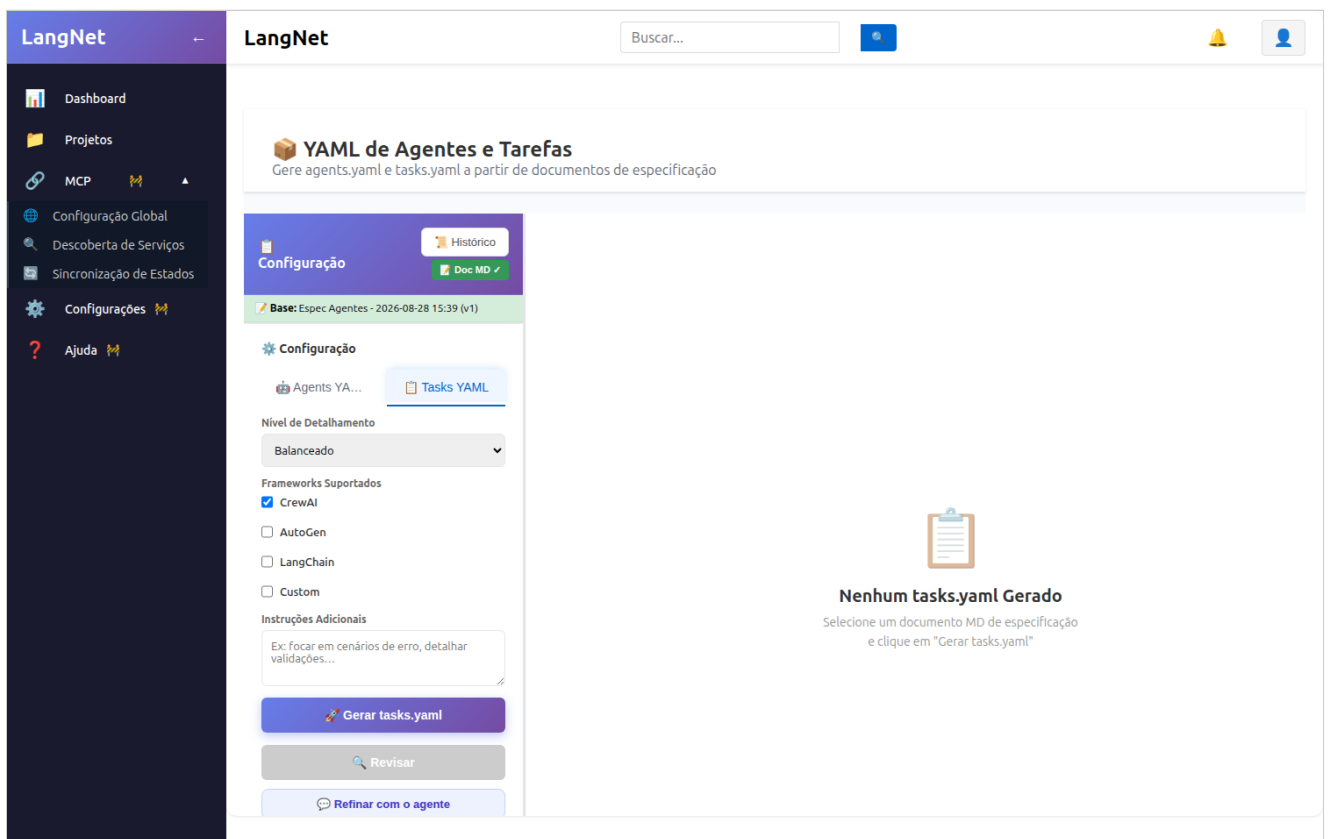
Etapa 3 — Modelo de Dados após refino (calculador + PostGIS)



Etapa 3.5 — UI Spec: telas ricas (mapa/metric-cards/gráfico)



Etapa 4 — ATS: especificação selecionada



Etapa 5 — tasks.yaml gerado (com calculate_ca_to)

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

Geração de Código

Geração do sistema agêntico Python (agents.yaml, tasks.yaml, tools.py, adapters.py, petri_net.json...) a partir das especificações. Refine com o agente, corrija warnings e execute o servidor.

Geração

Histórico

Nova geração

Sessão: code_gen_20260828_185616 (v1 - 81 arquivos - completed)

Configuração

SESSÕES (9)

code_gen_20260828_185616

v1 - 81 arquivos - completed

code_gen_20260827_161051

v1 - 85 arquivos - completed

code_gen_20260827_141244

v1 - 85 arquivos - completed

code_gen_20260827_140406

v1 - 85 arquivos - completed

Instruções Adicionais

Ex: focar em cenários de erro, detalhar validações...

Gerar Código

Refinar com o agente

ARQUIVOS (81)

ws-server/main.py

ws-server/websocket_server.py

ws-server/database_tool.py

ws-server/geoprocessamento_tool.py

ws-server/agis_bridge.py

ws-server/tools.py

ws-server/tools_std.py

ws-server/tools_ext.py

ws-server/adapters.py

ws-server/agents.yaml

ws-server/tasks.yaml

ws-server/petri_net.json

ws-server/requirements.txt

ws-server/.env.example

ws-server/Dockerfile

README.md

backend/Dockerfile

backend/main.py

backend/requirements.txt

ws-server/main.py

```
1 """
2 Uso do Solo v3 (regeneracao gerador corrigido) - entrypoint
3 Sobe o servidor WebSocket na porta 5025 que recebe execute_task
4 e dispara a task CrewAI correspondente.
5 """
6 import asyncio
7 import os
8 from dotenv import load_dotenv
9
10 from websocket_server import run_websocket_server
11
12 load_dotenv()
13
14 PROJECT_NAME = "Uso do Solo v3 (regeneracao gerador corrigido)"
15
16
17 def main():
18     port = int(os.getenv("WEBSOCKET_PORT", "5025"))
19     host = os.getenv("WEBSOCKET_HOST", "localhost")
20     print(f"🚀 {PROJECT_NAME} - WebSocket server em ws://{host}:{port}")
21     asyncio.run(run_websocket_server(host=host, port=port))
22
23
24 if __name__ == "__main__":
25     main()
26
```

Etapa 6 — Código gerado (81 arquivos)